

Presented by

TUYIRAMYE Christian

IGIRANEZA Justin

Face Attendance System

A real-time, contactless attendance solution using a webcam, OpenCV, and Flask.

The Problem



- Manual roll-call wastes 5 to 10 minutes every session.
- Paper sign-in and ID cards make buddy punching easy.
- Fingerprint scanners need shared contact hardware.
- Compiling weekly or monthly reports from paper logs is slow and error-prone.

Our Solution



- Look at the camera and you are checked in automatically.
- Look at the camera again later and you are checked out, hours computed.
- An HR dashboard for employees, attendance, leave and holidays.
- Runs on a standard laptop with a webcam. No GPU. No special hardware.

System Architecture

- Front end: HTML and JavaScript dashboard, or a Jupyter notebook UI.
- API layer: Flask serving twenty-four REST endpoints with CORS enabled.
- Recognition: OpenCV Haar cascade detection and an LBPH recognizer.
- Database: SQLAlchemy on SQLite for development, PostgreSQL for production.
- Logging: a rotating file handler writes to logs/app.log plus the console.

Face Recognition Pipeline

- 1 Capture a single 640 by 480 frame from the webcam.
- 2 Detect faces with a Haar cascade classifier.
- 3 Crop each face and resize to 200 by 200 grayscale.
- 4 Run the LBPH recognizer, which returns a label and a distance score.
- 5 Accept the match when the distance is below 70 (lower means a closer match).
- 6 Insert a new check-in record, or close an open one and compute working hours.

Data Model



- Employee: id, name, employee_id, department, role, encoding, is_active.
- AttendanceRecord: employee_id, date, check_in, check_out, hours, status.
- LeaveRequest: employee_id, leave_type, start_date, end_date, status.
- Holiday: name, date and description.

Key Features

- Live webcam preview with coloured bounding boxes for each face.
- Daily stats: total present, late, absent, on leave, average working hours.
- Monthly breakdown shown as a stacked bar chart.
- Per-employee summary including punctuality rate.
- CSV and JSON export for any date range.

Evaluation Results

Detection rate	98.5 %
Recognition accuracy	95.0 %
Mean latency per frame	168 ms
95th-percentile latency	240 ms
False acceptance rate	1.5 %
False rejection rate	3.5 %

Privacy and Security

- Only grayscale face crops are stored. No raw colour frames are kept.
- Database URL and secret key are read from environment variables.
- Soft delete is the default. A hard delete removes all stored face crops.
- Production checklist: HTTPS, admin authentication, retention policy, consent log.

Limitations



- No liveness detection yet, so a printed photo could potentially be accepted.
- LBPH accuracy drops under heavy backlight or large head turn angles.
- One camera per deployment in the current version.
- The browser version needs localhost or HTTPS for webcam access.

Future Work

- Add liveness detection through blink or 3D depth checks.
- Swap LBPH for FaceNet or ArcFace embeddings behind the same API.
- Build a mobile-first PWA for self-service check-in on a phone.
- Add anomaly alerts for repeated late check-ins or unusually short days.
- Support multiple cameras feeding into one back end.

How to Run



- Clone the repository and check out the branch `claude/epic-wozniak-t6J89`.
- Web version: `pip install -r requirements.txt`, then `python app.py`, then open `http://localhost:5000`.
- Notebook version: open `face_attendance.ipynb` in Jupyter and run all cells.
- Issues and pull requests are welcome.

Thank You

Questions are welcome.

TUYIRAMYE Christian & IGIRANEZA Justin

Independent Project - 2026